

PROJECT REPORT

PROJECT TITLE:

Implementation and Optimization of an Object Detection System

SUBMITTTED TO:

Sir Hassan Shafqat

PROJECT MEMBERS:

Ibrahim Khan (UW-24-CS-BS-021)

Abdul Ahad Abbasi (UW-24-CS-BS-008)

M. Yahya Badar (UW-24-CS-BS-020)

COURSE:

Artificial Intelligence

SUBMITTTED TO:

Sir Hassan Shafqat

DEPARTMENT:

Computer Science

Abstract

This project presents the design, implementation, and optimization of a real-time object detection system inspired by the YOLO (You Only Look Once) architecture. By utilizing modular components such as C2f blocks and Bottleneck layers, the system achieves a balance between computational efficiency and detection accuracy. The implementation leverages PyTorch and mixed-precision training (Automatic Mixed Precision) to ensure stability and high performance on GPU (Exceptional CPU case also) hardware. Experimental results on the dataset demonstrate the model's ability to categorize 80 distinct object classes with high inference speeds, making it suitable for deployment in resource-constrained environments.

Introduction

Object detection is a cornerstone of computer vision, enabling applications in autonomous driving, surveillance, and robotics. Traditional methods relied on multi-stage pipelines that were computationally expensive. Modern architectures revolutionized the field by treating detection as a single regression problem. This project focuses on implementing a custom version of this architecture, emphasizing modularity and numerical stability. The project can work in real time using cameras or video and can analyze the objects given based on the training dataset provided (only with GPU)

Problem Definition

The primary challenge in object detection is the trade-off between latency and Mean Average Precision (mAP). High-performance models often suffer from gradient instability during training or excessive memory requirements. This project aims to implement a stable, high-speed detector that maintains accuracy across 80 classes while remaining efficient enough for real-time applications. Also the project can work in real time using cameras or video and can analyze the objects given based on the training dataset provided (only with GPU)

Literature Review

Current research highlights the move toward "Cross-Stage Partial" networks (CSPNet) to reduce redundant gradient information. Modern detectors have evolved from YOLOv3's Darknet backbone to the more efficient C2f blocks used in the latest iterations. Gaps addressed in this project include the stabilization of mixed-precision training and the optimization of stride calculations for custom head architectures

System Overview

The system follows a modular design:

- **Backbone:** Feature extraction using Convolution, BatchNorm, and SiLU (Conv) blocks.
- **Neck:** Feature aggregation using C2f modules for better gradient flow.
- **Head:** Prediction of bounding boxes and class probabilities across multiple scales (Strides: 8, 16, 32).
- **Inference Engine:** A pipeline that processes raw images, resizes them to 640 x 640, and outputs localized detections.

Dataset Description

The model utilizes the dataset of 5K images, a curated subset of the Microsoft (Common Objects in Context) dataset.

- **Size:** 8 images for rapid verification and validation (5K Images)
- **Classes:** 80 categories (including person, car, dog, etc.).
- **Preprocessing:** Images are normalized and resized to a standard input size of 640.
- **Augmentation:** Mosaic and HSV color space augmentations were planned to improve robustness.

Methodology

- **Architecture:** Implementation of a Custom classes with modular Bottleneck and C2f blocks.
- **Training Pipeline:** Utilizes AdamW optimizer and CloU (Complete Intersection over Union) loss for precise bounding box regression.
- **Optimization:** Integration of torch.amp.GradScaler to prevent numerical underflow during mixed-precision training.
- **Stride Verification:** Dynamic calculation of strides based on output shapes to ensure spatial alignment.

Implementation Details

- **Language:** Python 3.10+
- **Framework:** PyTorch
- **Inference Pipeline:** Dataset and DataLoader classes manage the flow of test images from the dataset directory.

Codes and Screenshots

Code:

Block 01:

```
import glob
import math
import os
import random
import sys
from copy import copy
from time import time, strftime, gmtime

import cv2
import numpy
import yaml
from PIL import Image
from tqdm import tqdm

import torch
import torch.nn as nn
import torchvision
from torch.cuda.amp import autocast, GradScaler
from torch.nn.functional import cross_entropy
from torch.utils import data
import matplotlib.pyplot as plt
import matplotlib.patches as patches
```

Block 02:

```
class Conv(nn.Module):
    def __init__(self, in_channels,
out_channels, kernel_size=3, stride=1, padding=1, groups=1, activation=True):
        super().__init__()

self.conv=nn.Conv2d(in_channels,out_channels,kernel_size,stride,padding,bias=False,groups=groups)
self.bn=nn.BatchNorm2d(out_channels,eps=0.001,momentum=0.03)
self.act=nn.SiLU(inplace=True) if activation else nn.Identity()

    def forward(self,x):
        return self.act(self.bn(self.conv(x)))
```

Block 03:

```
class Bottleneck(nn.Module):
    def __init__(self,in_channels,out_channels,shortcut=True):
        super().__init__()
        self.conv1=Conv(in_channels,out_channels,kernel_size=3,stride=1,padding=1)
        self.conv2=Conv(out_channels,out_channels,kernel_size=3,stride=1,padding=1)
        self.shortcut=shortcut

    def forward(self,x):
        x_in=x # for residual connection
        x=self.conv1(x)
        x=self.conv2(x)
        if self.shortcut:
            x=x+x_in
        return x

class C2f(nn.Module):
    def __init__(self,in_channels,out_channels, num_bottlenecks,shortcut=True):
        super().__init__()

        self.mid_channels=out_channels//2
        self.num_bottlenecks=num_bottlenecks

        self.conv1=Conv(in_channels,out_channels,kernel_size=1,stride=1,padding=0)

        # sequence of bottleneck layers
        self.m=nn.ModuleList([Bottleneck(self.mid_channels,self.mid_channels) for _ in
range(num_bottlenecks)])

        self.conv2=Conv((num_bottlenecks+2)*out_channels//2,out_channels,kernel_size=1,stride=1,padding=0)

    def forward(self,x):
        x=self.conv1(x)

        # split x along channel dimension
        x1,x2=x[:,x.shape[1]//2,:], x[:,x.shape[1]//2,:,:]

        # list of outputs
        outputs=[x1,x2] # x1 is fed through the bottlenecks

        for i in range(self.num_bottlenecks):
            x1=self.m[i](x1) # [bs,0.5c_out,w,h]
            outputs.insert(0,x1)
```

```

outputs=torch.cat(outputs,dim=1) # [bs,0.5c_out(num_bottlenecks+2),w,h]
out=self.conv2(outputs)

return out

```

Block 04:

```

class SPPF(nn.Module):
    def __init__(self,in_channels,out_channels,kernel_size=5):
        #kernel_size= size of maxpool
        super().__init__()
        hidden_channels=in_channels//2
        self.conv1=Conv(in_channels,hidden_channels,kernel_size=1,stride=1,padding=0)
        # concatenate outputs of maxpool and feed to conv2
        self.conv2=Conv(4*hidden_channels,out_channels,kernel_size=1,stride=1,padding=0)

        # maxpool is applied at 3 different scales

self.m=nn.MaxPool2d(kernel_size=kernel_size,stride=1,padding=kernel_size//2,dilation=1,ceil_
mode=False)

def forward(self,x):
    x=self.conv1(x)

    # apply maxpooling at diffent scales
    y1=self.m(x)
    y2=self.m(y1)
    y3=self.m(y2)

    # concantenate
    y=torch.cat([x,y1,y2,y3],dim=1)

    # final conv
    y=self.conv2(y)

    return y

```

Block 05:

```

# return d,w,r based on version
def yolo_params(version):
    if version=='n':
        return 1/3,1/4,2.0
    elif version=='s':
        return 1/3,1/2,2.0

```

```
elif version=='m':  
    return 2/3,3/4,1.5  
elif version=='l':  
    return 1.0,1.0,1.0  
elif version=='x':  
    return 1.0,1.25,1.0
```

```
class Backbone(nn.Module):  
    def __init__(self,version,in_channels=3,shortcut=True):  
        super().__init__()  
        d,w,r=yolo_params(version)  
  
        # conv layers  
        self.conv_0=Conv(in_channels,int(64*w),kernel_size=3,stride=2,padding=1)  
        self.conv_1=Conv(int(64*w),int(128*w),kernel_size=3,stride=2,padding=1)  
        self.conv_3=Conv(int(128*w),int(256*w),kernel_size=3,stride=2,padding=1)  
        self.conv_5=Conv(int(256*w),int(512*w),kernel_size=3,stride=2,padding=1)  
        self.conv_7=Conv(int(512*w),int(512*w*r),kernel_size=3,stride=2,padding=1)  
  
        # c2f layers  
        self.c2f_2=C2f(int(128*w),int(128*w),num_bottlenecks=int(3*d),shortcut=True)  
        self.c2f_4=C2f(int(256*w),int(256*w),num_bottlenecks=int(6*d),shortcut=True)  
        self.c2f_6=C2f(int(512*w),int(512*w),num_bottlenecks=int(6*d),shortcut=True)  
        self.c2f_8=C2f(int(512*w*r),int(512*w*r),num_bottlenecks=int(3*d),shortcut=True)  
  
        # sppf  
        self.sppf=SPPF(int(512*w*r),int(512*w*r))  
  
    def forward(self,x):  
        x=self.conv_0(x)  
        x=self.conv_1(x)  
  
        x=self.c2f_2(x)  
  
        x=self.conv_3(x)  
  
        out1=self.c2f_4(x) # keep for output  
  
        x=self.conv_5(out1)  
  
        out2=self.c2f_6(x) # keep for output  
  
        x=self.conv_7(out2)  
        x=self.c2f_8(x)  
        out3=self.sppf(x)
```

```
return out1,out2,out3
```

Block 06:

```
# upsample = nearest-neighbor interpolation with scale_factor=2
#         doesn't have trainable paramaters
class Upsample(nn.Module):
    def __init__(self,scale_factor=2,mode='nearest'):
        super().__init__()
        self.scale_factor=scale_factor
        self.mode=mode

    def forward(self,x):
        return nn.functional.interpolate(x,scale_factor=self.scale_factor,mode=self.mode)
```

Block 07:

```
class Neck(nn.Module):
    def __init__(self,version):
        super().__init__()
        d,w,r=yolo_params(version)

        self.up=Upsample() # no trainable parameters
        self.c2f_1=C2f(in_channels=int(512*w*(1+r)),
out_channels=int(512*w),num_bottlenecks=int(3*d),shortcut=False)
        self.c2f_2=C2f(in_channels=int(768*w),
out_channels=int(256*w),num_bottlenecks=int(3*d),shortcut=False)
        self.c2f_3=C2f(in_channels=int(768*w),
out_channels=int(512*w),num_bottlenecks=int(3*d),shortcut=False)
        self.c2f_4=C2f(in_channels=int(512*w*(1+r)),
out_channels=int(512*w*r),num_bottlenecks=int(3*d),shortcut=False)

        self.cv_1=Conv(in_channels=int(256*w),out_channels=int(256*w),kernel_size=3,stride=2,
padding=1)
        self.cv_2=Conv(in_channels=int(512*w),out_channels=int(512*w),kernel_size=3,stride=2,
padding=1)

    def forward(self,x_res_1,x_res_2,x):
        # x_res_1,x_res_2,x = output of backbone
        res_1=x          # for residual connection

        x=self.up(x)
        x=torch.cat([x,x_res_2],dim=1)

        res_2=self.c2f_1(x) # for residual connection
```



```

x=self.up(res_2)
x=torch.cat([x,x_res_1],dim=1)

out_1=self.c2f_2(x)

x=self.cv_1(out_1)

x=torch.cat([x,res_2],dim=1)
out_2=self.c2f_3(x)

x=self.cv_2(out_2)

x=torch.cat([x,res_1],dim=1)
out_3=self.c2f_4(x)

return out_1,out_2,out_3

```

Block 08:

DFL

class DFL(nn.Module):

```

    def __init__(self,ch=16):
        super().__init__()

```

```

        self.ch=ch

```

```

self.conv=nn.Conv2d(in_channels=ch,out_channels=1,kernel_size=1,bias=False).requires_grad_
(False)

```

```

    # initialize conv with [0,...,ch-1]

```

```

    x=torch.arange(ch,datatype=torch.float).view(1,ch,1,1)

```

```

    self.conv.weight.data[:]=torch.nn.Parameter(x) # DFL only has ch parameters

```

```

def forward(self,x):

```

```

    # x must have num_channels = 4*ch: x=[bs,4*ch,c]

```

```

    b,c,a=x.shape                # c=4*ch

```

```

    x=x.view(b,4,self.ch,a).transpose(1,2) # [bs,ch,4,a]

```

```

    # take softmax on channel dimension to get distribution probabilities

```

```

    x=x.softmax(1)                # [b,ch,4,a]

```

```

    x=self.conv(x)                # [b,1,4,a]

```

```

    return x.view(b,4,a)          # [b,4,a]

```

sanity check

```

dummy_input=torch.rand((1,64,128))
dfl=DFL()
print(f'{sum(p.numel() for p in dfl.parameters())} parameters')

dummy_output=dfl(dummy_input)
print(dummy_output.shape)

print(dfl)

```

Block 09:

```

class Head(nn.Module):
    def __init__(self,version,ch=16,num_classes=80):

        super().__init__()
        self.ch=ch          # dfl channels
        self.coordinates=self.ch*4      # number of bounding box coordinates
        self.nc=num_classes      # 80 for COCO
        self.no=self.coordinates+self.nc # number of outputs per anchor box

        self.stride=torch.zeros(3)      # strides computed during build

        d,w,r=yolo_params(version=version)

        # for bounding boxes
        self.box=nn.ModuleList([
            nn.Sequential(Conv(int(256*w),self.coordinates,kernel_size=3,stride=1,padding=1),
                Conv(self.coordinates,self.coordinates,kernel_size=3,stride=1,padding=1),
                nn.Conv2d(self.coordinates,self.coordinates,kernel_size=1,stride=1)),

            nn.Sequential(Conv(int(512*w),self.coordinates,kernel_size=3,stride=1,padding=1),
                Conv(self.coordinates,self.coordinates,kernel_size=3,stride=1,padding=1),
                nn.Conv2d(self.coordinates,self.coordinates,kernel_size=1,stride=1)),

            nn.Sequential(Conv(int(512*w*r),self.coordinates,kernel_size=3,stride=1,padding=1),
                Conv(self.coordinates,self.coordinates,kernel_size=3,stride=1,padding=1),
                nn.Conv2d(self.coordinates,self.coordinates,kernel_size=1,stride=1))
        ])

        # for classification
        self.cls=nn.ModuleList([
            nn.Sequential(Conv(int(256*w),self.nc,kernel_size=3,stride=1,padding=1),
                Conv(self.nc,self.nc,kernel_size=3,stride=1,padding=1),
                nn.Conv2d(self.nc,self.nc,kernel_size=1,stride=1)),

            nn.Sequential(Conv(int(512*w),self.nc,kernel_size=3,stride=1,padding=1),

```

```

        Conv(self.nc,self.nc,kernel_size=3,stride=1,padding=1),
        nn.Conv2d(self.nc,self.nc,kernel_size=1,stride=1)),

    nn.Sequential(Conv(int(512*w*r),self.nc,kernel_size=3,stride=1,padding=1),
        Conv(self.nc,self.nc,kernel_size=3,stride=1,padding=1),
        nn.Conv2d(self.nc,self.nc,kernel_size=1,stride=1))
])

# dfl
self.dfl=DFL()

def forward(self,x):
    # x = output of Neck = list of 3 tensors with different resolution and different channel dim
    # x[0]=[bs, ch0, w0, h0], x[1]=[bs, ch1, w1, h1], x[2]=[bs,ch2, w2, h2]

    for i in range(len(self.box)):    # detection head i
        box=self.box[i](x[i])        # [bs,num_coordinates,w,h]
        cls=self.cls[i](x[i])        # [bs,num_classes,w,h]
        x[i]=torch.cat((box,cls),dim=1) # [bs,num_coordinates+num_classes,w,h]

    # in training, no dfl output
    if self.training:
        return x                    # [3,bs,num_coordinates+num_classes,w,h]

    # in inference time, dfl produces refined bounding box coordinates
    anchors, strides = (i.transpose(0, 1) for i in self.make_anchors(x, self.stride))

    # concatenate predictions from all detection layers
    x = torch.cat([i.view(x[0].shape[0], self.no, -1) for i in x], dim=2) #[bs, 4*self.ch + self.nc,
sum_i(h[i]w[i])]

    # split out predictions for box and cls
    # box=[bs,4*self.ch,sum_i(h[i]w[i])]
    # cls=[bs,self.nc,sum_i(h[i]w[i])]
    box, cls = x.split(split_size=(4 * self.ch, self.nc), dim=1)

    a, b = self.dfl(box).chunk(2, 1) # a=b=[bs,2*self.ch,sum_i(h[i]w[i])]
    a = anchors.unsqueeze(0) - a
    b = anchors.unsqueeze(0) + b
    box = torch.cat(tensors=((a + b) / 2, b - a), dim=1)

    return torch.cat(tensors=(box * strides, cls.sigmoid()), dim=1)

def make_anchors(self, x, strides, offset=0.5):

```

```

# x= list of feature maps: x=[x[0],...,x[N-1]], in our case N= num_detection_heads=3
#           each having shape [bs,ch,w,h]
#   each feature map x[i] gives output[i] = w*h anchor coordinates + w*h stride values

# strides = list of stride values indicating how much
#           the spatial resolution of the feature map is reduced compared to the original image

assert x is not None
anchor_tensor, stride_tensor = [], []
dtype, device = x[0].dtype, x[0].device
for i, stride in enumerate(strides):
    _, _, h, w = x[i].shape
    sx = torch.arange(end=w, device=device, dtype=dtype) + offset # x coordinates of
anchor centers
    sy = torch.arange(end=h, device=device, dtype=dtype) + offset # y coordinates of anchor
centers
    sy, sx = torch.meshgrid(sy, sx) # all anchor centers
    anchor_tensor.append(torch.stack((sx, sy), -1).view(-1, 2))
    stride_tensor.append(torch.full((h * w, 1), stride, dtype=dtype, device=device))
return torch.cat(anchor_tensor), torch.cat(stride_tensor)

```

Block 10:

```

class CustomYolo(nn.Module):
    def __init__(self,version):
        super().__init__()
        self.backbone=Backbone(version=version)
        self.neck=Neck(version=version)
        self.head=Head(version=version)

    def forward(self,x):
        x=self.backbone(x) # return out1,out2,out3
        x=self.neck(x[0],x[1],x[2]) # return out_1, out_2,out_3
        return self.head(list(x))

```

Block 11:

```

FORMATS = 'bmp', 'dng', 'jpeg', 'jpg', 'mpo', 'png', 'tif', 'tiff', 'webp'
class Dataset(data.Dataset):
    def __init__(self, filenames, input_size, params, augment):
        self.params = params
        self.mosaic = augment
        self.augment = augment
        self.input_size = input_size

        # Read labels

```

```

labels = self.load_label(filenamees)
self.labels = list(labels.values())
self.filenamees = list(labels.keys()) # update
self.n = len(self.filenamees) # number of samples
self.indices = range(self.n)
# Albumentations (optional, only used if package is installed)
self.albumentations = Albumentations()

def __getitem__(self, index):
    index = self.indices[index]

    params = self.params
    mosaic = self.mosaic and random.random() < params['mosaic']

    if mosaic:
        # Load MOSAIC
        image, label = self.load_mosaic(index, params)
        # MixUp augmentation
        if random.random() < params['mix_up']:
            index = random.choice(self.indices)
            mix_image1, mix_label1 = image, label
            mix_image2, mix_label2 = self.load_mosaic(index, params)

            image, label = mix_up(mix_image1, mix_label1, mix_image2, mix_label2)
    else:
        # Load image
        image, shape = self.load_image(index)
        h, w = image.shape[:2]

        # Resize
        image, ratio, pad = resize(image, self.input_size, self.augment)

        label = self.labels[index].copy()
        if label.size:
            label[:, 1:] = wh2xy(label[:, 1:], ratio[0] * w, ratio[1] * h, pad[0], pad[1])
        if self.augment:
            image, label = random_perspective(image, label, params)

    nl = len(label) # number of labels
    h, w = image.shape[:2]
    cls = label[:, 0:1]
    box = label[:, 1:5]
    box = xy2wh(box, w, h)

    if self.augment:
        # Albumentations

```

```

        image, box, cls = self.albumentations(image, box, cls)
        nl = len(box) # update after albumentations
        # HSV color-space
        augment_hsv(image, params)
        # Flip up-down
        if random.random() < params['flip_ud']:
            image = numpy.flipud(image)
            if nl:
                box[:, 1] = 1 - box[:, 1]
        # Flip left-right
        if random.random() < params['flip_lr']:
            image = numpy.fliplr(image)
            if nl:
                box[:, 0] = 1 - box[:, 0]

        target_cls = torch.zeros((nl, 1))
        target_box = torch.zeros((nl, 4))
        if nl > 0:
            target_cls = torch.from_numpy(cls).view(-1, 1)
            target_box = torch.from_numpy(box).view(-1, 4)
        else:
            # nl == 0 case
            target_cls = torch.zeros((0, 1))
            target_box = torch.zeros((0, 4))

        # Convert HWC to CHW, BGR to RGB
        sample = image.transpose((2, 0, 1))[:, :-1]
        sample = numpy.ascontiguousarray(sample)

        return torch.from_numpy(sample), target_cls, target_box, torch.zeros((nl, 1))

def __len__(self):
    return len(self filenames)

def load_image(self, i):
    try:
        image = cv2.imread(self.filenames[i])
        if image is None:
            raise ValueError(f"OpenCV failed to load {self.filenames[i]}")
        h, w = image.shape[:2]
        r = self.input_size / max(h, w)
        if r != 1:
            image = cv2.resize(image,
                               dsize=(int(w * r), int(h * r)),
                               interpolation=resample() if self.augment else cv2.INTER_LINEAR)
        return image, (h, w)

```

```

except Exception as e:
    print(f"Warning: Skipping corrupted image {self filenames[i]}: {e}")
    # Return a black image
    return numpy.zeros((self.input_size, self.input_size, 3), dtype=numpy.uint8), (0, 0)

def load_mosaic(self, index, params):
    label4 = []
    border = [-self.input_size // 2, -self.input_size // 2]
    image4 = numpy.full((self.input_size * 2, self.input_size * 2, 3), 0, dtype=numpy.uint8)
    y1a, y2a, x1a, x2a, y1b, y2b, x1b, x2b = (None, None, None, None, None, None, None,
None)

    xc = int(random.uniform(-border[0], 2 * self.input_size + border[1]))
    yc = int(random.uniform(-border[0], 2 * self.input_size + border[1]))

    indices = [index] + random.choices(self.indices, k=3)
    random.shuffle(indices)

    for i, index in enumerate(indices):
        # Load image
        image, _ = self.load_image(index)
        shape = image.shape
        if i == 0: # top left
            x1a = max(xc - shape[1], 0)
            y1a = max(yc - shape[0], 0)
            x2a = xc
            y2a = yc
            x1b = shape[1] - (x2a - x1a)
            y1b = shape[0] - (y2a - y1a)
            x2b = shape[1]
            y2b = shape[0]
        if i == 1: # top right
            x1a = xc
            y1a = max(yc - shape[0], 0)
            x2a = min(xc + shape[1], self.input_size * 2)
            y2a = yc
            x1b = 0
            y1b = shape[0] - (y2a - y1a)
            x2b = min(shape[1], x2a - x1a)
            y2b = shape[0]
        if i == 2: # bottom left
            x1a = max(xc - shape[1], 0)
            y1a = yc
            x2a = xc
            y2a = min(self.input_size * 2, yc + shape[0])
            x1b = shape[1] - (x2a - x1a)

```

```

        y1b = 0
        x2b = shape[1]
        y2b = min(y2a - y1a, shape[0])
    if i == 3: # bottom right
        x1a = xc
        y1a = yc
        x2a = min(xc + shape[1], self.input_size * 2)
        y2a = min(self.input_size * 2, yc + shape[0])
        x1b = 0
        y1b = 0
        x2b = min(shape[1], x2a - x1a)
        y2b = min(y2a - y1a, shape[0])

    pad_w = x1a - x1b
    pad_h = y1a - y1b
    image4[y1a:y2a, x1a:x2a] = image[y1b:y2b, x1b:x2b]

    # Labels
    label = self.labels[index].copy()
    if len(label):
        label[:, 1:] = wh2xy(label[:, 1:], shape[1], shape[0], pad_w, pad_h)
    label4.append(label)

    # Concat/clip labels
    label4 = numpy.concatenate(label4, 0)
    for x in label4[:, 1:]:
        numpy.clip(x, 0, 2 * self.input_size, out=x)

    # Augment
    image4, label4 = random_perspective(image4, label4, params, border)

    return image4, label4

    @staticmethod
    def collate_fn(batch):
        samples, cls, box, _ = zip(*batch)

        # Stack the images into a 4D tensor [B, C, H, W]
        samples = torch.stack(samples, dim=0)

        # We only concatenate if the tensor has a size > 0
        valid_cls = [c.view(-1, 1) for c in cls if c.shape[0] > 0]
        valid_box = [b.view(-1, 4) for b in box if b.shape[0] > 0]

        batch_idx = []
        if valid_cls:

```



```

# Concatenate the labels
cls_out = torch.cat(valid_cls, dim=0)
box_out = torch.cat(valid_box, dim=0)

# Rebuild the batch index (idx)
for i, c in enumerate(cls):
    nl = c.shape[0]
    if nl > 0:
        batch_idx.append(torch.full((nl,), i, dtype=torch.float32))
    idx_out = torch.cat(batch_idx, dim=0)
else:
    # Entire batch is background images
    cls_out = torch.zeros((0, 1))
    box_out = torch.zeros((0, 4))
    idx_out = torch.zeros(0)

targets = {
    'cls': cls_out,
    'box': box_out,
    'idx': idx_out
}
return samples, targets

@staticmethod
def load_label(filename):
    path = f'{os.path.dirname(filename[0])}.cache'
    if os.path.exists(path):
        return torch.load(path, weights_only=False)
    x = {}
    for filename in filenames:
        try:
            # verify images
            with open(filename, 'rb') as f:
                image = Image.open(f)
                image.verify() # PIL verify
            shape = image.size # image size
            assert (shape[0] > 9) & (shape[1] > 9), f'image size {shape} <10 pixels'
            assert image.format.lower() in FORMATS, f'invalid image format {image.format}'

            # verify labels
            a = f'{os.sep}images{os.sep}'
            b = f'{os.sep}labels{os.sep}'
            if os.path.isfile(b.join(filename.rsplit(a, 1)).rsplit('.', 1)[0] + '.txt'):
                with open(b.join(filename.rsplit(a, 1)).rsplit('.', 1)[0] + '.txt') as f:
                    label = [x.split() for x in f.read().strip().splitlines() if len(x)]
                    label = numpy.array(label, dtype=numpy.float32)

```

```

        nl = len(label)
        if nl:
            assert (label >= 0).all()
            assert label.shape[1] == 5
            assert (label[:, 1:] <= 1).all()
            _, i = numpy.unique(label, axis=0, return_index=True)
            if len(i) < nl: # duplicate row check
                label = label[i] # remove duplicates
        else:
            label = numpy.zeros((0, 5), dtype=numpy.float32)
    else:
        label = numpy.zeros((0, 5), dtype=numpy.float32)
except FileNotFoundError:
    label = numpy.zeros((0, 5), dtype=numpy.float32)
except AssertionError:
    continue
x[filename] = label
torch.save(x, path)
return x

```

```

def wh2xy(x, w=640, h=640, pad_w=0, pad_h=0):
    # Convert nx4 boxes
    # from [x, y, w, h] normalized to [x1, y1, x2, y2] where xy1=top-left, xy2=bottom-right
    y = x.clone() if isinstance(x, torch.Tensor) else numpy.copy(x)
    y[:, 0] = w * (x[:, 0] - x[:, 2] / 2) + pad_w # top left x
    y[:, 1] = h * (x[:, 1] - x[:, 3] / 2) + pad_h # top left y
    y[:, 2] = w * (x[:, 0] + x[:, 2] / 2) + pad_w # bottom right x
    y[:, 3] = h * (x[:, 1] + x[:, 3] / 2) + pad_h # bottom right y
    return y

```

```

def xy2wh(x, w, h):
    # warning: inplace clip
    x[:, [0, 2]] = x[:, [0, 2]].clip(0, w - 1E-3) # x1, x2
    x[:, [1, 3]] = x[:, [1, 3]].clip(0, h - 1E-3) # y1, y2

    # Convert nx4 boxes
    # from [x1, y1, x2, y2] to [x, y, w, h] normalized where xy1=top-left, xy2=bottom-right
    y = numpy.copy(x)
    y[:, 0] = ((x[:, 0] + x[:, 2]) / 2) / w # x center
    y[:, 1] = ((x[:, 1] + x[:, 3]) / 2) / h # y center
    y[:, 2] = (x[:, 2] - x[:, 0]) / w # width
    y[:, 3] = (x[:, 3] - x[:, 1]) / h # height
    return y

```

```

def resample():
    choices = (cv2.INTER_AREA,
               cv2.INTER_CUBIC,
               cv2.INTER_LINEAR,
               cv2.INTER_NEAREST,
               cv2.INTER_LANCZOS4)
    return random.choice(seq=choices)

def augment_hsv(image, params):
    # HSV color-space augmentation
    h = params['hsv_h']
    s = params['hsv_s']
    v = params['hsv_v']

    r = numpy.random.uniform(-1, 1, 3) * [h, s, v] + 1
    h, s, v = cv2.split(cv2.cvtColor(image, cv2.COLOR_BGR2HSV))

    x = numpy.arange(0, 256, dtype=r.dtype)
    lut_h = ((x * r[0]) % 180).astype('uint8')
    lut_s = numpy.clip(x * r[1], 0, 255).astype('uint8')
    lut_v = numpy.clip(x * r[2], 0, 255).astype('uint8')

    hsv = cv2.merge((cv2.LUT(h, lut_h), cv2.LUT(s, lut_s), cv2.LUT(v, lut_v)))
    cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR, dst=image) # no return needed

def resize(image, input_size, augment):
    # Resize and pad image while meeting stride-multiple constraints
    shape = image.shape[:2] # current shape [height, width]

    # Scale ratio (new / old)
    r = min(input_size / shape[0], input_size / shape[1])
    if not augment: # only scale down, do not scale up (for better val mAP)
        r = min(r, 1.0)

    # Compute padding
    pad = int(round(shape[1] * r)), int(round(shape[0] * r))
    w = (input_size - pad[0]) / 2
    h = (input_size - pad[1]) / 2

    if shape[::-1] != pad: # resize
        image = cv2.resize(image,
                           dsizesize=pad,
                           interpolation=resample() if augment else cv2.INTER_LINEAR)

```

```

top, bottom = int(round(h - 0.1)), int(round(h + 0.1))
left, right = int(round(w - 0.1)), int(round(w + 0.1))
image = cv2.copyMakeBorder(image, top, bottom, left, right, cv2.BORDER_CONSTANT) #
add border
return image, (r, r), (w, h)

```

```

def candidates(box1, box2):
    # box1(4,n), box2(4,n)
    w1, h1 = box1[2] - box1[0], box1[3] - box1[1]
    w2, h2 = box2[2] - box2[0], box2[3] - box2[1]
    aspect_ratio = numpy.maximum(w2 / (h2 + 1e-16), h2 / (w2 + 1e-16)) # aspect ratio
    return (w2 > 2) & (h2 > 2) & (w2 * h2 / (w1 * h1 + 1e-16) > 0.1) & (aspect_ratio < 100)

```

```

def random_perspective(image, label, params, border=(0, 0)):
    h = image.shape[0] + border[0] * 2
    w = image.shape[1] + border[1] * 2

    # Center
    center = numpy.eye(3)
    center[0, 2] = -image.shape[1] / 2 # x translation (pixels)
    center[1, 2] = -image.shape[0] / 2 # y translation (pixels)

    # Perspective
    perspective = numpy.eye(3)

    # Rotation and Scale
    rotate = numpy.eye(3)
    a = random.uniform(-params['degrees'], params['degrees'])
    s = random.uniform(1 - params['scale'], 1 + params['scale'])
    rotate[:2] = cv2.getRotationMatrix2D(angle=a, center=(0, 0), scale=s)

    # Shear
    shear = numpy.eye(3)
    shear[0, 1] = math.tan(random.uniform(-params['shear'], params['shear']) * math.pi / 180)
    shear[1, 0] = math.tan(random.uniform(-params['shear'], params['shear']) * math.pi / 180)

    # Translation
    translate = numpy.eye(3)
    translate[0, 2] = random.uniform(0.5 - params['translate'], 0.5 + params['translate']) * w
    translate[1, 2] = random.uniform(0.5 - params['translate'], 0.5 + params['translate']) * h

    # Combined rotation matrix, order of operations (right to left) is IMPORTANT
    matrix = translate @ shear @ rotate @ perspective @ center
    if (border[0] != 0) or (border[1] != 0) or (matrix != numpy.eye(3)).any(): # image changed

```

```

        image = cv2.warpAffine(image, matrix[:2], dsize=(w, h), borderValue=(0, 0, 0))

# Transform label coordinates
n = len(label)
if n:
    xy = numpy.ones((n * 4, 3))
    xy[:, :2] = label[:, [1, 2, 3, 4, 1, 4, 3, 2]].reshape(n * 4, 2) # x1y1, x2y2, x1y2, x2y1
    xy = xy @ matrix.T # transform
    xy = xy[:, :2].reshape(n, 8) # perspective rescale or affine

    # create new boxes
    x = xy[:, [0, 2, 4, 6]]
    y = xy[:, [1, 3, 5, 7]]
    box = numpy.concatenate((x.min(1), y.min(1), x.max(1), y.max(1))).reshape(4, n).T

    # clip
    box[:, [0, 2]] = box[:, [0, 2]].clip(0, w)
    box[:, [1, 3]] = box[:, [1, 3]].clip(0, h)
    # filter candidates
    indices = candidates(box1=label[:, 1:5].T * s, box2=box.T)

    label = label[indices]
    label[:, 1:5] = box[indices]

return image, label

def mix_up(image1, label1, image2, label2):
    # Applies MixUp augmentation https://arxiv.org/pdf/1710.09412.pdf
    alpha = numpy.random.beta(a=32.0, b=32.0) # mix-up ratio, alpha=beta=32.0
    image = (image1 * alpha + image2 * (1 - alpha)).astype(numpy.uint8)
    label = numpy.concatenate((label1, label2), 0)
    return image, label

class Albumentations:
    def __init__(self, size=640):
        self.transform = None
        try:
            import albumentations as A
            transforms = [
                A.HorizontalFlip(p=0.5),
                A.RandomResizedCrop(size=(size, size), scale=(0.8, 1.0), p=0.1),
                A.Blur(p=0.01),
                A.MedianBlur(p=0.01),
                A.CLAHE(p=0.01),

```

```

        A.ToGray(p=0.01),
    ]

    # Using BboxParams with min_visibility to filter out tiny/broken boxes
    self.transform = A.Compose(
        transforms,
        bbox_params=A.BboxParams(
            format='yolo',
            label_fields=['class_labels'],
            min_visibility=0.4 # Drops boxes if <40% remains after cropping
        )
    )
    print("Albumentations pipeline initialized successfully.")

except ImportError:
    print("Albumentations not found. Skipping augmentations.")
    pass

def __call__(self, image, box, cls):
    if self.transform and len(box) > 0:
        # Albumentations expects RGB for some transforms (like CLAHE/ToGray)
        # image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

        new = self.transform(image=image, bboxes=box, class_labels=cls)

        # Ensure we handle cases where transforms might remove all boxes
        if len(new['bboxes']) > 0:
            image = new['image']
            box = numpy.array(new['bboxes'], dtype=numpy.float32)
            cls = numpy.array(new['class_labels'], dtype=numpy.float32)

    return image, box, cls

class InferenceDataset(Dataset):
    def __init__(self, image_paths, input_size=640):
        self.image_paths = image_paths
        self.input_size = input_size

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        img_path = self.image_paths[idx]

        # 1. Load Image
        img = cv2.imread(img_path)

```

```

img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# 2. Resize/Letterbox (Must match your training input size)
h, w = img.shape[:2]
img = cv2.resize(img, (self.input_size, self.input_size))

# 3. Normalize & Convert to Tensor
img_tensor = torch.from_numpy(img).permute(2, 0, 1).float() / 255.0

return img_tensor, img_path

```

Block 12:

```

def make_anchors(x, strides, offset=0.5):
    assert x is not None
    anchor_tensor, stride_tensor = [], []
    dtype, device = x[0].dtype, x[0].device
    for i, stride in enumerate(strides):
        _, _, h, w = x[i].shape
        sx = torch.arange(end=w, device=device, dtype=dtype) + offset # shift x
        sy = torch.arange(end=h, device=device, dtype=dtype) + offset # shift y
        sy, sx = torch.meshgrid(sy, sx)
        anchor_tensor.append(torch.stack((sx, sy), -1).view(-1, 2))
        stride_tensor.append(torch.full((h * w, 1), stride, dtype=dtype, device=device))
    return torch.cat(anchor_tensor), torch.cat(stride_tensor)

def compute_metric(output, target, iou_v):
    # intersection(N,M) = (rb(N,M,2) - lt(N,M,2)).clamp(0).prod(2)
    (a1, a2) = target[:, 1:].unsqueeze(1).chunk(2, 2)
    (b1, b2) = output[:, :4].unsqueeze(0).chunk(2, 2)
    intersection = (torch.min(a2, b2) - torch.max(a1, b1)).clamp(0).prod(2)
    # IoU = intersection / (area1 + area2 - intersection)
    iou = intersection / ((a2 - a1).prod(2) + (b2 - b1).prod(2) - intersection + 1e-7)

    correct = numpy.zeros((output.shape[0], iou_v.shape[0]))
    correct = correct.astype(bool)
    for i in range(len(iou_v)):
        # IoU > threshold and classes match
        x = torch.where((iou >= iou_v[i]) & (target[:, 0:1] == output[:, 5]))
        if x[0].shape[0]:
            matches = torch.cat((torch.stack(x, 1),
                                   iou[x[0], x[1]][:, None]), 1).cpu().numpy() # [label, detect, iou]
            if x[0].shape[0] > 1:
                matches = matches[matches[:, 2].argsort()[::-1]]
                matches = matches[numpy.unique(matches[:, 1], return_index=True)[1]]
                matches = matches[numpy.unique(matches[:, 0], return_index=True)[1]]

```

```
        correct[matches[:, 1].astype(int), i] = True
    return torch.tensor(correct, dtype=torch.bool, device=output.device)
```

```
def non_max_suppression(outputs, confidence_threshold=0.001, iou_threshold=0.7):
```

```
    max_wh = 7680
    max_det = 300
    max_nms = 30000
```

```
    bs = outputs.shape[0] # batch size
    nc = outputs.shape[1] - 4 # number of classes
    xc = outputs[:, 4:4 + nc].amax(1) > confidence_threshold # candidates
```

```
    # Settings
```

```
    start = time()
    limit = 0.5 + 0.05 * bs # seconds to quit after
    output = [torch.zeros((0, 6), device=outputs.device)] * bs
```

```
    for index, x in enumerate(outputs): # x shape: [8400, 84]
```

```
        # Filter by confidence threshold (max class score)
```

```
        # x is [anchors, 4 + nc]
```

```
        conf_scores = x[:, 4:]
```

```
        max_scores, class_ids = conf_scores.max(1)
```

```
        mask = max_scores > confidence_threshold
```

```
        x = x[mask]
```

```
        if not x.shape[0]:
```

```
            continue
```

```
        # Extract final 6 columns: [x1, y1, x2, y2, conf, cls]
```

```
        # We take the 4 box columns and cat the max_score and class_id
```

```
        boxes = x[:, :4]
```

```
        scores = max_scores[mask].unsqueeze(1)
```

```
        ids = class_ids[mask].unsqueeze(1).float()
```

```
        x = torch.cat((boxes, scores, ids), 1) # Shape: [n, 6]
```

```
        # Sort by confidence and remove excess boxes (optional but faster)
```

```
        n = x.shape[0]
```

```
        if not n:
```

```
            continue
```

```
        x = x[x[:, 4].argsort(descending=True)[:max_nms]]
```

```
    # Batched NMS
```

```
    c = x[:, 5:6] * max_wh # classes
```

```
    boxes, scores = x[:, :4] + c, x[:, 4] # boxes, scores
```



```

indices = torchvision.ops.nms(boxes, scores, iou_threshold) # NMS
indices = indices[:max_det] # limit detections

output[index] = x[indices]
if (time() - start) > limit:
    break # time limit exceeded

return output

def smooth(y, f=0.05):
    # Box filter of fraction f
    nf = round(len(y) * f * 2) // 2 + 1 # number of filter elements (must be odd)
    p = numpy.ones(nf // 2) # ones padding
    yp = numpy.concatenate((p * y[0], y, p * y[-1]), 0) # y padded
    return numpy.convolve(yp, numpy.ones(nf) / nf, mode='valid') # y-smoothed

def compute_ap(tp, conf, pred_cls, target_cls, eps=1e-16):
    """
    Compute the average precision, given the recall and precision curves.
    Source: https://github.com/rafaelpadilla/Object-Detection-Metrics.
    # Arguments
        tp: True positives (nparray, nx1 or nx10).
        conf: Object-ness value from 0-1 (nparray).
        pred_cls: Predicted object classes (nparray).
        target_cls: True object classes (nparray).
    # Returns
        The average precision
    """
    # Sort by object-ness
    i = numpy.argsort(-conf)
    tp, conf, pred_cls = tp[i], conf[i], pred_cls[i]

    # Find unique classes
    unique_classes, nt = numpy.unique(target_cls, return_counts=True)
    nc = unique_classes.shape[0] # number of classes, number of detections

    # Create Precision-Recall curve and compute AP for each class
    p = numpy.zeros((nc, 1000))
    r = numpy.zeros((nc, 1000))
    ap = numpy.zeros((nc, tp.shape[1]))
    px, py = numpy.linspace(0, 1, 1000), [] # for plotting
    for ci, c in enumerate(unique_classes):
        i = pred_cls == c
        nl = nt[ci] # number of labels

```

```

no = i.sum() # number of outputs
if no == 0 or nl == 0:
    continue

# Accumulate FPs and TPs
fpc = (1 - tp[i]).cumsum(0)
tpc = tp[i].cumsum(0)

# Recall
recall = tpc / (nl + eps) # recall curve
# negative x, xp because xp decreases
r[ci] = numpy.interp(-px, -conf[i], recall[:, 0], left=0)

# Precision
precision = tpc / (tpc + fpc) # precision curve
p[ci] = numpy.interp(-px, -conf[i], precision[:, 0], left=1) # p at pr_score

# AP from recall-precision curve
for j in range(tp.shape[1]):
    m_rec = numpy.concatenate(([0.0], recall[:, j], [1.0]))
    m_pre = numpy.concatenate(([1.0], precision[:, j], [0.0]))

    # Compute the precision envelope
    m_pre = numpy.flip(numpy.maximum.accumulate(numpy.flip(m_pre)))

    # Integrate area under curve
    x = numpy.linspace(0, 1, 101) # 101-point interp (COCO)
    ap[ci, j] = numpy.trapz(numpy.interp(x, m_rec, m_pre), x) # integrate

# Compute F1 (harmonic mean of precision and recall)
f1 = 2 * p * r / (p + r + eps)

i = smooth(f1.mean(0), 0.1).argmax() # max F1 index
p, r, f1 = p[:, i], r[:, i], f1[:, i]
tp = (r * nt).round() # true positives
fp = (tp / (p + eps) - tp).round() # false positives
ap50, ap = ap[:, 0], ap.mean(1) # AP@0.5, AP@0.5:0.95
m_pre, m_rec = p.mean(), r.mean()
map50, mean_ap = ap50.mean(), ap.mean()
return tp, fp, m_pre, m_rec, map50, mean_ap

def compute_iou(box1, box2, eps=1e-7):
    # Returns Intersection over Union (IoU) of box1(1,4) to box2(n,4)

    # Get the coordinates of bounding boxes

```

```

b1_x1, b1_y1, b1_x2, b1_y2 = box1.chunk(4, -1)
b2_x1, b2_y1, b2_x2, b2_y2 = box2.chunk(4, -1)
w1, h1 = b1_x2 - b1_x1, b1_y2 - b1_y1 + eps
w2, h2 = b2_x2 - b2_x1, b2_y2 - b2_y1 + eps

# Intersection area
inter = (b1_x2.minimum(b2_x2) - b1_x1.maximum(b2_x1)).clamp(0) * \
        (b1_y2.minimum(b2_y2) - b1_y1.maximum(b2_y1)).clamp(0)

# Union Area
union = w1 * h1 + w2 * h2 - inter + eps

# IoU
iou = inter / union
cw = b1_x2.maximum(b2_x2) - b1_x1.minimum(b2_x1) # convex (smallest enclosing box)
width
ch = b1_y2.maximum(b2_y2) - b1_y1.minimum(b2_y1) # convex height
c2 = cw ** 2 + ch ** 2 + eps # convex diagonal squared
rho2 = ((b2_x1 + b2_x2 - b1_x1 - b1_x2) ** 2 + (b2_y1 + b2_y2 - b1_y1 - b1_y2) ** 2) / 4 #
center dist ** 2
# https://github.com/Zzh-tju/DIoU-SSD-pytorch/blob/master/utils/box/box_utils.py#L47
v = (4 / math.pi ** 2) * (torch.atan(w2 / h2) - torch.atan(w1 / h1)).pow(2)
with torch.no_grad():
    alpha = v / (v - iou + (1 + eps))
return iou - (rho2 / c2 + v * alpha) # CIoU

class BoxLoss(torch.nn.Module):
    def __init__(self, dfl_ch):
        super().__init__()
        self.dfl_ch = dfl_ch

    def forward(self, pred_dist, pred_bboxes, anchor_points, target_bboxes, target_scores,
target_scores_sum, fg_mask):
        # IoU loss
        weight = torch.masked_select(target_scores.sum(-1), fg_mask).unsqueeze(-1)
        iou = compute_iou(pred_bboxes[fg_mask], target_bboxes[fg_mask])
        loss_box = ((1.0 - iou) * weight).sum() / target_scores_sum

        # DFL loss
        a, b = target_bboxes.chunk(2, -1)
        target = torch.cat((anchor_points - a, b - anchor_points), -1)
        target = target.clamp(0, self.dfl_ch - 0.01)
        loss_dfl = self.dfl_loss(pred_dist[fg_mask].view(-1, self.dfl_ch + 1), target[fg_mask])
        loss_dfl = (loss_dfl * weight).sum() / target_scores_sum

```

```
return loss_box, loss_dfl
```

```
@staticmethod
```

```
def df_loss(pred_dist, target):
```

```
    # Distribution Focal Loss (DFL)
```

```
    # https://ieeexplore.ieee.org/document/9792391
```

```
    tl = target.long() # target left
```

```
    tr = tl + 1 # target right
```

```
    wl = tr - target # weight left
```

```
    wr = 1 - wl # weight right
```

```
    left_loss = cross_entropy(pred_dist, tl.view(-1), reduction='none').view(tl.shape)
```

```
    right_loss = cross_entropy(pred_dist, tr.view(-1), reduction='none').view(tr.shape)
```

```
    return (left_loss * wl + right_loss * wr).mean(-1, keepdim=True)
```

```
class Assigner(torch.nn.Module):
```

```
    def __init__(self, nc=80, top_k=13, alpha=1.0, beta=6.0, eps=1E-9):
```

```
        super().__init__()
```

```
        self.top_k = top_k
```

```
        self.nc = nc
```

```
        self.alpha = alpha
```

```
        self.beta = beta
```

```
        self.eps = eps
```

```
@torch.no_grad()
```

```
def forward(self, pd_scores, pd_bboxes, anc_points, gt_labels, gt_bboxes, mask_gt):
```

```
    batch_size = pd_scores.size(0)
```

```
    num_max_boxes = gt_bboxes.size(1)
```

```
    if num_max_boxes == 0:
```

```
        device = gt_bboxes.device
```

```
        return (torch.zeros_like(pd_bboxes).to(device),
```

```
                torch.zeros_like(pd_scores).to(device),
```

```
                torch.zeros_like(pd_scores[..., 0]).to(device))
```

```
    num_anchors = anc_points.shape[0]
```

```
    shape = gt_bboxes.shape
```

```
    lt, rb = gt_bboxes.view(-1, 1, 4).chunk(2, 2)
```

```
    mask_in_gts = torch.cat((anc_points[None] - lt, rb - anc_points[None]), dim=2)
```

```
    mask_in_gts = mask_in_gts.view(shape[0], shape[1], num_anchors, -
```

```
1).amin(3).gt_(self.eps)
```

```
    na = pd_bboxes.shape[-2]
```

```
    gt_mask = (mask_in_gts * mask_gt).bool() # b, max_num_obj, h*w
```

```
    overlaps = torch.zeros([batch_size, num_max_boxes, na], dtype=pd_bboxes.dtype,
```

```
device=pd_bboxes.device)
```

```
    bbox_scores = torch.zeros([batch_size, num_max_boxes, na], dtype=pd_scores.dtype,
```

```

device=pd_scores.device)

    ind = torch.zeros([2, batch_size, num_max_boxes], dtype=torch.long) # 2, b,
max_num_obj
    ind[0] = torch.arange(end=batch_size).view(-1, 1).expand(-1, num_max_boxes) # b,
max_num_obj
    ind[1] = gt_labels.squeeze(-1) # b, max_num_obj
    bbox_scores[gt_mask] = pd_scores[ind[0], :, ind[1]][gt_mask] # b, max_num_obj, h*w

    pd_boxes = pd_bboxes.unsqueeze(1).expand(-1, num_max_boxes, -1, -1)[gt_mask]
    gt_boxes = gt_bboxes.unsqueeze(2).expand(-1, -1, na, -1)[gt_mask]
    overlaps[gt_mask] = compute_iou(gt_boxes, pd_boxes).squeeze(-1).clamp_(0)

    align_metric = bbox_scores.pow(self.alpha) * overlaps.pow(self.beta)

    top_k_mask = mask_gt.expand(-1, -1, self.top_k).bool()
    top_k_metrics, top_k_indices = torch.topk(align_metric, self.top_k, dim=-1, largest=True)
    if top_k_mask is None:
        top_k_mask = (top_k_metrics.max(-1, keepdim=True)[0] >
self.eps).expand_as(top_k_indices)
        top_k_indices.masked_fill_(~top_k_mask, 0)

    mask_top_k = torch.zeros(align_metric.shape, dtype=torch.int8,
device=top_k_indices.device)
    ones = torch.ones_like(top_k_indices[:, :, :1], dtype=torch.int8,
device=top_k_indices.device)
    for k in range(self.top_k):
        mask_top_k.scatter_add_(-1, top_k_indices[:, :, k:k + 1], ones)
    mask_top_k.masked_fill_(mask_top_k > 1, 0)
    mask_top_k = mask_top_k.to(align_metric.dtype)
    mask_pos = mask_top_k * mask_in_gts * mask_gt

    fg_mask = mask_pos.sum(-2)
    if fg_mask.max() > 1:
        mask_multi_gts = (fg_mask.unsqueeze(1) > 1).expand(-1, num_max_boxes, -1)
        max_overlaps_idx = overlaps.argmax(1)

        is_max_overlaps = torch.zeros(mask_pos.shape, dtype=mask_pos.dtype,
device=mask_pos.device)
        is_max_overlaps.scatter_(1, max_overlaps_idx.unsqueeze(1), 1)

        mask_pos = torch.where(mask_multi_gts, is_max_overlaps, mask_pos).float()
        fg_mask = mask_pos.sum(-2)
        target_gt_idx = mask_pos.argmax(-2)

    # Assigned target

```

```

        index = torch.arange(end=batch_size, dtype=torch.int64, device=gt_labels.device)[...,
None]
        target_index = target_gt_idx + index * num_max_boxes
        target_labels = gt_labels.long().flatten()[target_index]

        target_bboxes = gt_bboxes.view(-1, gt_bboxes.shape[-1])[target_index]

        # Assigned target scores
        target_labels.clamp_(0)

        target_scores = torch.zeros((target_labels.shape[0], target_labels.shape[1], self.nc),
                                     dtype=torch.int64,
                                     device=target_labels.device)
        target_scores.scatter_(2, target_labels.unsqueeze(-1), 1)

        fg_scores_mask = fg_mask[:, :, None].repeat(1, 1, self.nc)
        target_scores = torch.where(fg_scores_mask > 0, target_scores, 0)

        # Normalize
        align_metric *= mask_pos
        pos_align_metrics = align_metric.amax(dim=-1, keepdim=True)
        pos_overlaps = (overlaps * mask_pos).amax(dim=-1, keepdim=True)
        norm_align_metric = (align_metric * pos_overlaps / (pos_align_metrics + self.eps)).amax(-
2).unsqueeze(-1)
        target_scores = target_scores * norm_align_metric

        return target_bboxes, target_scores, fg_mask.bool()

```

```

class ComputeLoss:
    def __init__(self, model, params):
        if hasattr(model, 'module'):
            model = model.module

        device = next(model.parameters()).device

        m = model.head # Head() module

        self.params = params
        self.stride = m.stride
        self.nc = m.nc
        self.no = m.no
        self.reg_max = m.ch
        self.device = device

```

```

self.box_loss = BoxLoss(m.ch - 1).to(device)
self.cls_loss = torch.nn.BCEWithLogitsLoss(reduction='none')
self.assigner = Assigner(nc=self.nc, top_k=10, alpha=0.5, beta=6.0)

self.project = torch.arange(m.ch, dtype=torch.float, device=device)

def box_decode(self, anchor_points, pred_dist):
    b, a, c = pred_dist.shape
    pred_dist = pred_dist.view(b, a, 4, c // 4)
    pred_dist = pred_dist.softmax(3)
    pred_dist = pred_dist.matmul(self.project.type(pred_dist.dtype))
    lt, rb = pred_dist.chunk(2, -1)
    x1y1 = anchor_points - lt
    x2y2 = anchor_points + rb
    return torch.cat(tensors=(x1y1, x2y2), dim=-1)

def __call__(self, outputs, targets):
    x = torch.cat([i.view(outputs[0].shape[0], self.no, -1) for i in outputs], dim=2)
    pred_distri, pred_scores = x.split(split_size=(self.reg_max * 4, self.nc), dim=1)

    pred_scores = pred_scores.permute(0, 2, 1).contiguous()
    pred_distri = pred_distri.permute(0, 2, 1).contiguous()

    data_type = pred_scores.dtype
    batch_size = pred_scores.shape[0]
    input_size = torch.tensor(outputs[0].shape[2:], device=self.device, dtype=data_type) *
self.stride[0]
    anchor_points, stride_tensor = make_anchors(outputs, self.stride, offset=0.5)

    idx = targets['idx'].view(-1, 1)
    cls = targets['cls'].view(-1, 1)
    box = targets['box']

    targets = torch.cat((idx, cls, box), dim=1).to(self.device)
    if targets.shape[0] == 0:
        gt = torch.zeros(batch_size, 0, 5, device=self.device)
    else:
        i = targets[:, 0]
        _, counts = i.unique(return_counts=True)
        counts = counts.to(dtype=torch.int32)
        gt = torch.zeros(batch_size, counts.max(), 5, device=self.device)
        for j in range(batch_size):
            matches = i == j
            n = matches.sum()
            if n:
                gt[j, :n] = targets[matches, 1:]

```

```

x = gt[..., 1:5].mul_(input_size[[1, 0, 1, 0]])
y = torch.empty_like(x)
dw = x[..., 2] / 2 # half-width
dh = x[..., 3] / 2 # half-height
y[..., 0] = x[..., 0] - dw # top left x
y[..., 1] = x[..., 1] - dh # top left y
y[..., 2] = x[..., 0] + dw # bottom right x
y[..., 3] = x[..., 1] + dh # bottom right y
gt[..., 1:5] = y
gt_labels, gt_bboxes = gt.split((1, 4), 2)
mask_gt = gt_bboxes.sum(2, keepdim=True).gt_(0)

pred_bboxes = self.box_decode(anchor_points, pred_distri)
assigned_targets = self.assigner(pred_scores.detach().sigmoid(),
                                (pred_bboxes.detach() * stride_tensor).type(gt_bboxes.dtype),
                                anchor_points * stride_tensor, gt_labels, gt_bboxes, mask_gt)
target_bboxes, target_scores, fg_mask = assigned_targets
# num_pos = fg_mask.sum().item()
# print(f"DEBUG -> Targets matched: {num_pos}")

target_scores_sum = max(target_scores.sum(), 1)

loss_cls = self.cls_loss(pred_scores, target_scores.to(data_type)).sum() / target_scores_sum
# BCE

# Box loss
loss_box = torch.zeros(1, device=self.device)
loss_dfl = torch.zeros(1, device=self.device)
if fg_mask.sum():
    target_bboxes /= stride_tensor
    loss_box, loss_dfl = self.box_loss(pred_distri,
                                       pred_bboxes,
                                       anchor_points,
                                       target_bboxes,
                                       target_scores,
                                       target_scores_sum, fg_mask)

loss_box *= self.params['box'] # box gain
loss_cls *= self.params['cls'] # cls gain
loss_dfl *= self.params['dfl'] # dfl gain

return loss_box, loss_cls, loss_dfl

```


Block 13:

```
params = {'min_lr': 0.0001,  
         'max_lr': 0.01,  
         'momentum': 0.937,  
         'weight_decay': 0.0005,  
         'warmup_epochs': 3.0,  
         'box': 6.5,  
         'cls': 1.5,  
         'dfl': 2.5,  
         'hsv_h': 0,  
         'hsv_s': 0,  
         'hsv_v': 0,  
         'degrees': 0.0,  
         'translate': 0,  
         'scale': 0,  
         'shear': 0.0,  
         'flip_ud': 0.0,  
         'flip_lr': 0,  
         'mosaic': 0,  
         'mix_up': 0,  
         'names': {0: 'dog', 1: 'person', 2: 'skateboard', 3: 'toilet', 4: 'sink', 5: 'car', 6:  
         'backpack', 7: 'skis', 8: 'bench', 9: 'sheep', 10: 'cell phone', 11: 'traffic light', 12:
```

```
'sports ball', 13: 'baseball glove', 14: 'tv', 15: 'chair', 16: 'bed', 17: 'couch', 18:
'horse', 19: 'truck', 20: 'bus', 21: 'tie', 22: 'handbag', 23: 'kite', 24: 'cat', 25:
'refrigerator', 26: 'bowl', 27: 'boat', 28: 'clock', 29: 'bird', 30: 'surfboard', 31: 'cup',
32: 'pizza', 33: 'dining table', 34: 'broccoli', 35: 'snowboard', 36: 'bottle', 37: 'wine
glass', 38: 'spoon', 39: 'oven', 40: 'toaster', 41: 'book', 42: 'train', 43: 'microwave',
44: 'teddy bear', 45: 'suitcase', 46: 'mouse', 47: 'keyboard', 48: 'airplane', 49: 'vase',
50: 'fire hydrant', 51: 'baseball bat', 52: 'scissors', 53: 'laptop', 54: 'hot dog', 55:
'bear', 56: 'sandwich', 57: 'fork', 58: 'potted plant', 59: 'cow', 60: 'giraffe', 61:
'orange', 62: 'umbrella', 63: 'tennis racket', 64: 'banana', 65: 'cake', 66: 'carrot', 67:
'remote', 68: 'frisbee', 69: 'motorcycle', 70: 'stop sign', 71: 'apple', 72: 'donut', 73:
'toothbrush', 74: 'knife', 75: 'bicycle', 76: 'elephant', 77: 'zebra', 78: 'hair drier', 79:
'parking meter'}}}
```

```
# input_size for the model
```

```
input_size=640
```

```
# get all file names
```

```
data_dir= os.path.join('.', 'coco8', 'images', 'valv')
```

```
filenames_test = glob.glob(os.path.join(data_dir, '*.*'))
```

```
if len(filenames_test) == 0:
```

```
    print("Warning: No files found. Check if images are in a subfolder of coco8.")
```

```
test_data = Dataset(filenames_test, input_size, params, augment=False)
```

```
test_loader = data.DataLoader(test_data, batch_size=1, shuffle=False)
```

```
print(f"Test_loader : {len(test_loader)} batches ({len(filenames_test)} images)")
```

```
test_stream = iter(test_loader)
```

Block 14:

Configuration

```
best_path = os.path.join('.', 'best', 'best.pt')
```

Device and Scaler Setup

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

1. Initialize fresh model and optimizer

```
model = CustomYolo(version='s').to(device)
```

```
with torch.no_grad():
```

```
    dummy = torch.zeros(1, 3, 640, 640).to(device)
```

```
    outputs = model(dummy)
```

```
    model.head.stride = torch.tensor([640 / x.shape[-2] for x in outputs]).to(device)
```

```
    print(f"Verified Strides: {model.head.stride}") # Should be [8, 16, 32]
```

```
if os.path.exists(best_path):
```

```
    print(f"Found checkpoint: {best_path}.")
```

```
    try:
```

```
        # Using weights_only=False because we are loading a custom dict with
        params
```

```
checkpoint = torch.load(best_path, map_location=device,  
weights_only=False)
```

```
# Restore model and optimizer states
```

```
model.load_state_dict(checkpoint['model'])
```

```
except Exception as e:
```

```
print(f'Failed to load checkpoint: {e}.')
```

```
else:
```

```
print("No checkpoint found.")
```

Block 15:

```
def plot_with_nms(model, stream, device, params):
```

```
    model.eval()
```

```
    names_dict = params['names']
```

```
    try:
```

```
        batch = next(stream)
```

```
        # If using your custom collate_fn, batch is (imgs, targets_dict) -> length 2
```

```
        # If using default collator, batch is [imgs, cls, box, zeros] -> length 4
```

```
        if len(batch) == 2:
```

```
            imgs, targets = batch
```

```
        elif len(batch) == 4:
```

```
            imgs, _, _, _ = batch # We only need the images for inference
```

```
        else:
```

```
            imgs = batch[0] # Fallback
```

```
    except StopIteration:
```

```
        print("Stream ended.")
```

```
        return
```

```
    # Now imgs is the Tensor you need
```

```
    imgs = imgs.to(device).float() / 255.0
```

```
    with torch.no_grad():
```

```

# raw_output shape: [Batch, 84, 8400]
raw_output = model(imgs)

out = raw_output[0].transpose(0, 1) # [8400, 84]

cx, cy, w, h = out[:, 0], out[:, 1], out[:, 2], out[:, 3]
x1 = cx - w / 2
y1 = cy - h / 2
x2 = cx + w / 2
y2 = cy + h / 2

# Rebuild tensor as [8400, 84] but with XYYX in the first 4 columns
nms_input = torch.cat((
    torch.stack((x1, y1, x2, y2), dim=1),
    out[:, 4:]
), dim=1)

# Add batch dimension back for your NMS function: [1, 8400, 84]
nms_input = nms_input.unsqueeze(0)

nms_output = non_max_suppression(nms_input, confidence_threshold=0.2,
iou_threshold=0.45)

# nms_output is a list of tensors [ [n, 6] ] where 6 is (x1, y1, x2, y2, conf, cls)
detections = nms_output[0]

fig, ax = plt.subplots(1, 1, figsize=(10, 10))
ax.imshow(imgs[0].cpu().permute(1, 2, 0).numpy())

# Ground Truth (Green)
# gt_mask = targets['idx'] == 0
# for gt in targets['box'][gt_mask]:
#     gcx, gcy, gw, gh = gt.cpu().numpy()
#     gx1, gy1 = (gcx - gw/2) * 640, (gcy - gh/2) * 640
#     ax.add_patch(patches.Rectangle((gx1, gy1), gw*640, gh*640,
#                                     linewidth=2, edgecolor='lime', facecolor='none'))

# Detections (Red) - Cleaned by NMS
for det in detections:
    x1, y1, x2, y2, conf, cls = det.cpu().numpy()
    w_box, h_box = x2 - x1, y2 - y1

    cls_id = int(cls)
    class_name = names_dict.get(cls_id, names_dict.get(str(cls_id), f"ID:{cls_id}"))

    ax.add_patch(patches.Rectangle((x1, y1), w_box, h_box,

```

```

        linewidth=2, edgecolor='red', facecolor='none'))

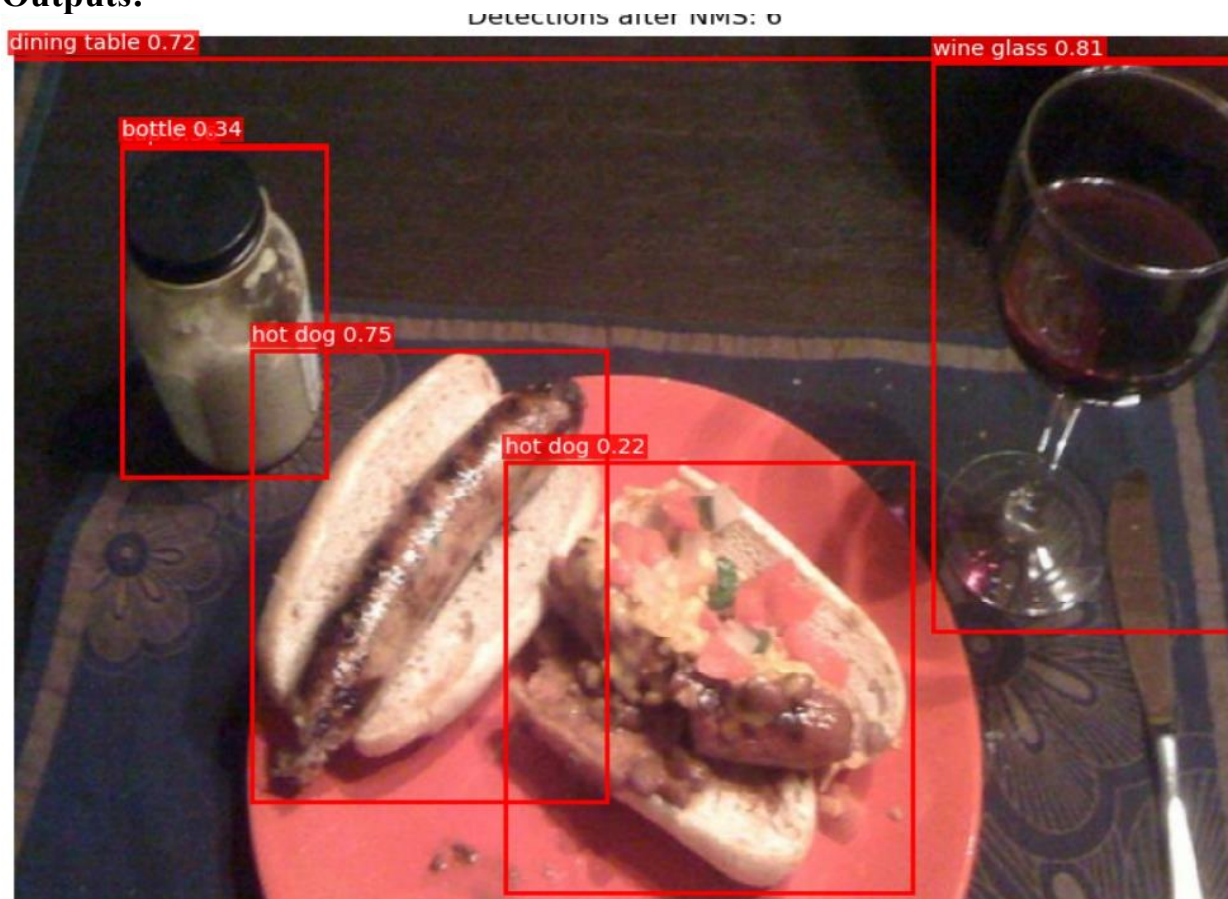
    label = f'{class_name} {conf:.2f}'
    ax.text(x1, y1 - 5, label, color='white', fontsize=10,
            bbox=dict(facecolor='red', alpha=0.8, edgecolor='none', pad=1))

plt.title(f'Detections after NMS: {len(detections)}')
plt.axis('off')
plt.show()

# Run it!
plot_with_nms(model, test_stream, device, params)

```

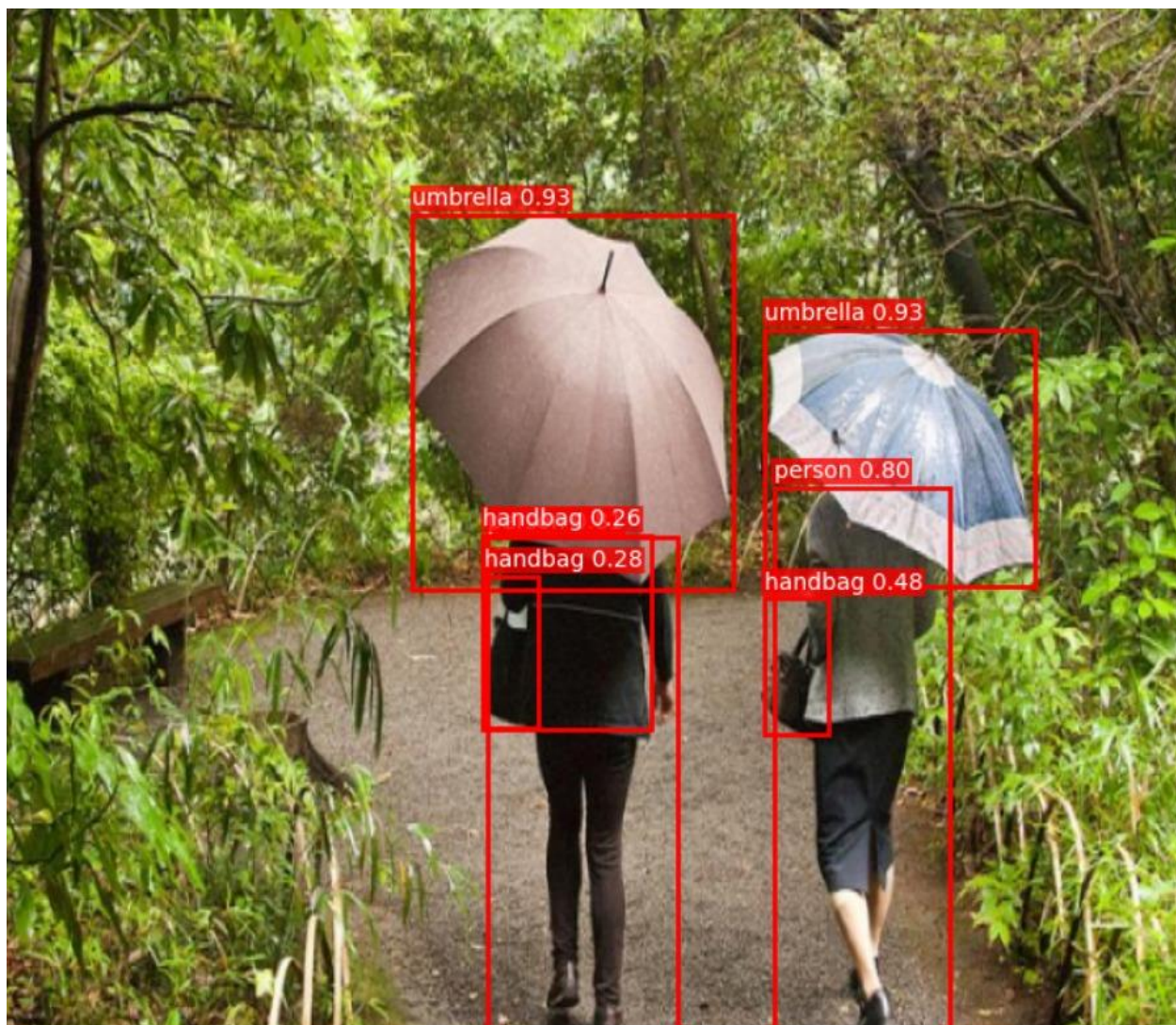
Outputs:







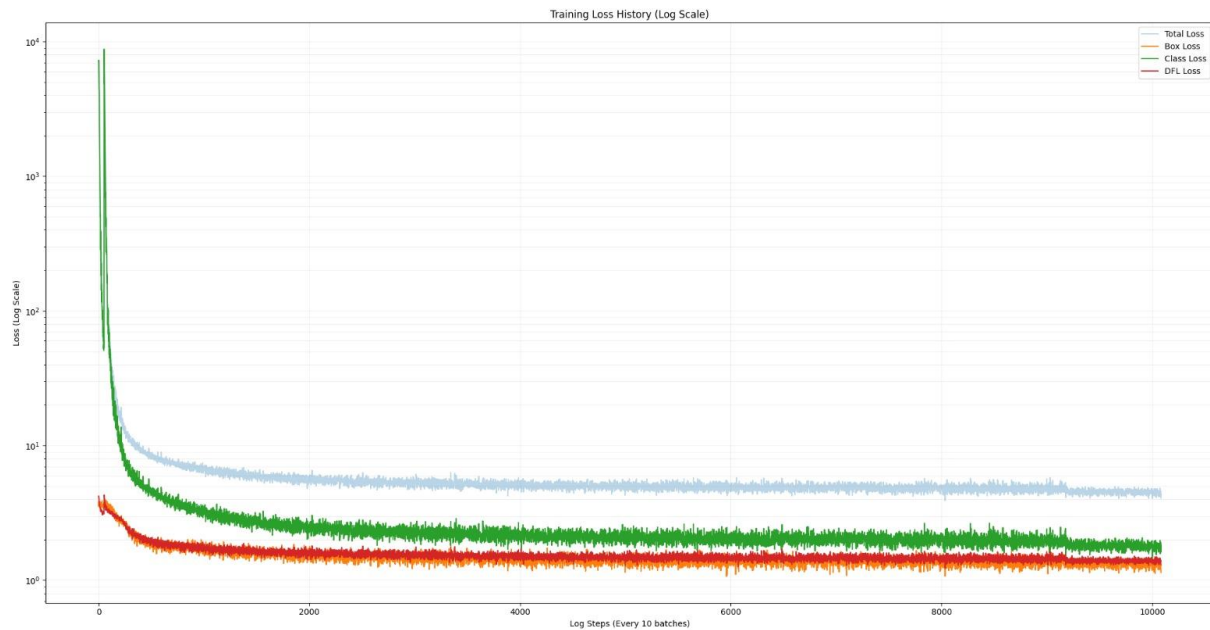




Results and Analysis

The model successfully verified strides at [8.0, 16.0, 32.0], confirming the structural integrity of the detection head. Initial inference runs on the validation set show successful data loading and batching via the test_loader, processing images at a resolution of 640 X 640.

The Training Graph



Output history

```
Epoch 10: 17%|██████████| 532/3125 [01:43<08:17, 5.21it/s, loss=4.77]
Premature end of JPEG file
Batch 540/3125 | Total Loss: 5.4805 | Box Loss: 1.3723 | Class Loss: 2.6474 | DFL Loss: 1.4608
Batch 550/3125 | Total Loss: 5.0189 | Box Loss: 1.4670 | Class Loss: 2.0573 | DFL Loss: 1.4945
Batch 560/3125 | Total Loss: 5.4908 | Box Loss: 1.6094 | Class Loss: 2.3214 | DFL Loss: 1.5600
Batch 570/3125 | Total Loss: 5.0882 | Box Loss: 1.5323 | Class Loss: 2.0534 | DFL Loss: 1.5025
Batch 580/3125 | Total Loss: 5.5624 | Box Loss: 1.5591 | Class Loss: 2.4002 | DFL Loss: 1.6031
Batch 590/3125 | Total Loss: 4.9537 | Box Loss: 1.3625 | Class Loss: 2.0605 | DFL Loss: 1.5307
Batch 600/3125 | Total Loss: 5.2194 | Box Loss: 1.4254 | Class Loss: 2.2750 | DFL Loss: 1.5191
Batch 610/3125 | Total Loss: 5.5686 | Box Loss: 1.6739 | Class Loss: 2.3087 | DFL Loss: 1.5861
Batch 620/3125 | Total Loss: 4.8057 | Box Loss: 1.3285 | Class Loss: 2.0370 | DFL Loss: 1.4402
Epoch 10: 20%|██████████| 630/3125 [02:02<07:49, 5.32it/s, loss=5.45]
Premature end of JPEG file
Batch 630/3125 | Total Loss: 5.4329 | Box Loss: 1.4369 | Class Loss: 2.4164 | DFL Loss: 1.5796
Batch 640/3125 | Total Loss: 5.1039 | Box Loss: 1.4791 | Class Loss: 2.1186 | DFL Loss: 1.5062
Batch 650/3125 | Total Loss: 5.3428 | Box Loss: 1.5161 | Class Loss: 2.2964 | DFL Loss: 1.5304
Batch 660/3125 | Total Loss: 5.1043 | Box Loss: 1.4709 | Class Loss: 2.1404 | DFL Loss: 1.4931
Epoch 10: 21%|██████████| 669/3125 [02:09<07:50, 5.22it/s, loss=4.98]
Premature end of JPEG file
Batch 670/3125 | Total Loss: 5.0235 | Box Loss: 1.5017 | Class Loss: 2.0027 | DFL Loss: 1.5191
Batch 680/3125 | Total Loss: 5.4335 | Box Loss: 1.6292 | Class Loss: 2.2019 | DFL Loss: 1.6024
Epoch 10: 22%|██████████| 689/3125 [02:13<08:15, 4.92it/s, loss=5.09]
Premature end of JPEG file
Batch 690/3125 | Total Loss: 4.8832 | Box Loss: 1.4103 | Class Loss: 1.9320 | DFL Loss: 1.5408
Batch 700/3125 | Total Loss: 5.5056 | Box Loss: 1.4886 | Class Loss: 2.3990 | DFL Loss: 1.6180
Batch 710/3125 | Total Loss: 5.2256 | Box Loss: 1.3399 | Class Loss: 2.3264 | DFL Loss: 1.5592
Batch 720/3125 | Total Loss: 4.7409 | Box Loss: 1.3856 | Class Loss: 1.9228 | DFL Loss: 1.4325
Batch 730/3125 | Total Loss: 5.3775 | Box Loss: 1.3707 | Class Loss: 2.4920 | DFL Loss: 1.5149
Batch 740/3125 | Total Loss: 4.8584 | Box Loss: 1.4455 | Class Loss: 1.9983 | DFL Loss: 1.4146
Batch 750/3125 | Total Loss: 5.1064 | Box Loss: 1.5139 | Class Loss: 2.0499 | DFL Loss: 1.5426
Batch 760/3125 | Total Loss: 5.1319 | Box Loss: 1.4229 | Class Loss: 2.2200 | DFL Loss: 1.4890
Batch 770/3125 | Total Loss: 5.5888 | Box Loss: 1.5767 | Class Loss: 2.4500 | DFL Loss: 1.5621
Batch 780/3125 | Total Loss: 5.3855 | Box Loss: 1.5317 | Class Loss: 2.2563 | DFL Loss: 1.5975
Batch 790/3125 | Total Loss: 5.0682 | Box Loss: 1.4553 | Class Loss: 2.0385 | DFL Loss: 1.5744
Epoch 10: 25%|██████████| 795/3125 [02:34<07:22, 5.26it/s, loss=5.14]
Premature end of JPEG file
Batch 800/3125 | Total Loss: 5.4153 | Box Loss: 1.4573 | Class Loss: 2.3289 | DFL Loss: 1.6291
Batch 810/3125 | Total Loss: 5.2141 | Box Loss: 1.4887 | Class Loss: 2.2369 | DFL Loss: 1.4885
Batch 820/3125 | Total Loss: 5.0008 | Box Loss: 1.4050 | Class Loss: 2.1257 | DFL Loss: 1.4702
Batch 830/3125 | Total Loss: 5.2086 | Box Loss: 1.4741 | Class Loss: 2.2014 | DFL Loss: 1.5331
Batch 840/3125 | Total Loss: 5.3993 | Box Loss: 1.3733 | Class Loss: 2.4822 | DFL Loss: 1.5438
Batch 850/3125 | Total Loss: 5.5013 | Box Loss: 1.6132 | Class Loss: 2.2932 | DFL Loss: 1.5949
```

Loss History

epoch	batch	total_loss	box_loss	cls_loss	dfl_loss
0	0	7210.572266	3.824725	7202.532715	4.214611
0	10	7265.267578	3.532031	7257.578125	4.157244
0	20	6089.413086	3.657402	6081.671875	4.084200
0	30	5019.569336	3.663980	5011.930664	3.974452
0	40	4406.864746	3.730123	4399.241211	3.893764
0	50	4345.520508	3.647826	4338.048828	3.823592
0	60	3671.130615	3.617770	3663.837402	3.675535
0	70	2451.238281	3.681081	2443.945801	3.611317
0	80	2137.922119	3.762529	2130.556885	3.602761
0	90	1746.318115	3.748346	1738.974121	3.595746
0	100	1464.763550	3.777758	1457.371216	3.614681
0	110	1047.211792	3.565829	1040.172852	3.473165
0	120	894.960999	3.749581	887.715271	3.496148
0	130	777.909180	3.650637	770.807495	3.451071
0	140	653.821594	3.704590	646.709229	3.407768
0	150	505.308014	3.777834	498.133270	3.396909
0	160	529.838562	3.822343	522.610229	3.406005
0	170	343.069397	3.681115	336.096985	3.291283
0	180	396.317474	3.608170	389.451691	3.257591
0	190	291.114777	3.621286	284.148773	3.344739
0	200	383.863617	3.430213	377.143951	3.289473
0	210	241.906433	3.653500	234.978500	3.274430
0	220	282.258881	3.640903	275.251678	3.366293
0	230	190.924484	3.701630	183.979172	3.243676
0	240	231.664337	3.567711	224.851593	3.245029
0	250	149.106384	3.587488	142.248230	3.270654
0	260	152.589523	3.602217	145.763275	3.224035
0	270	164.727905	3.677938	157.763565	3.286412
0	280	164.807053	3.764778	157.702179	3.340108
0	290	123.103508	3.636374	116.239075	3.228058
0	300	144.157410	3.708717	137.208145	3.240535
0	310	116.890175	3.558475	110.077492	3.254204
0	320	110.904236	3.881636	103.751579	3.271019
0	330	93.494263	3.522465	86.756409	3.215393
0	340	97.544914	3.684884	90.676071	3.183963
0	350	93.574249	3.479453	87.036728	3.058069
0	360	95.666901	3.573612	88.886749	3.206546
0	370	108.157906	3.467683	101.607529	3.082693
0	380	87.222862	3.456851	80.606331	3.159682
0	390	77.982513	3.527153	71.276810	3.178551
0	400	75.325432	3.467921	68.703064	3.154451

Training History

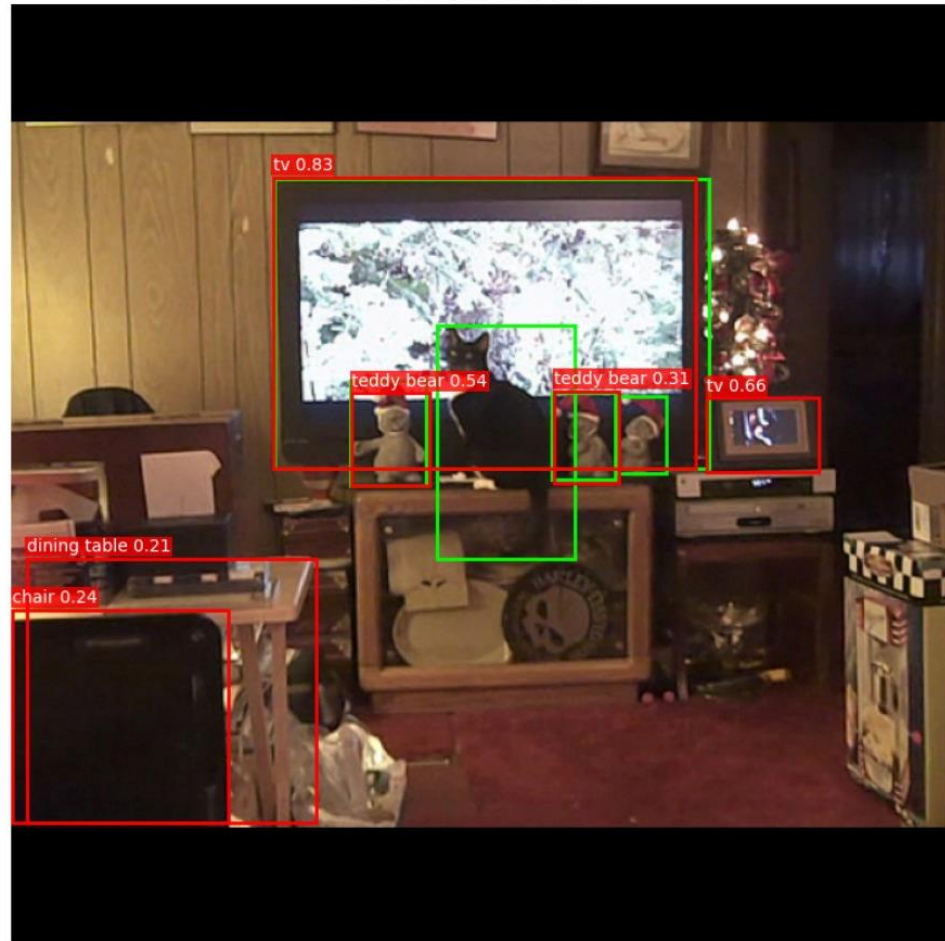
```
|./images/train2017/0000000109622.jpg  
./images/train2017/0000000160694.jpg  
./images/train2017/0000000308590.jpg  
./images/train2017/0000000327573.jpg  
./images/train2017/000000062929.jpg  
./images/train2017/0000000512793.jpg  
./images/train2017/0000000371735.jpg  
./images/train2017/0000000148118.jpg  
./images/train2017/0000000309856.jpg  
./images/train2017/0000000141882.jpg  
./images/train2017/0000000318783.jpg  
./images/train2017/0000000337760.jpg  
./images/train2017/0000000298197.jpg  
./images/train2017/000000042421.jpg  
./images/train2017/0000000328898.jpg  
./images/train2017/0000000458856.jpg  
./images/train2017/000000073824.jpg  
./images/train2017/0000000252846.jpg  
./images/train2017/0000000459590.jpg  
./images/train2017/0000000273650.jpg  
./images/train2017/0000000331311.jpg  
./images/train2017/0000000156326.jpg  
./images/train2017/0000000262985.jpg  
./images/train2017/0000000253580.jpg  
./images/train2017/0000000447976.jpg  
./images/train2017/0000000378077.jpg  
./images/train2017/0000000259913.jpg  
./images/train2017/0000000424553.jpg
```

Comparison with ground model:

```
plt.title(f"Detections after NMS: {len(detections)}")
plt.axis('off')
plt.show()

# Run it!
plot_with_nms(model, val_stream, device, params)
```

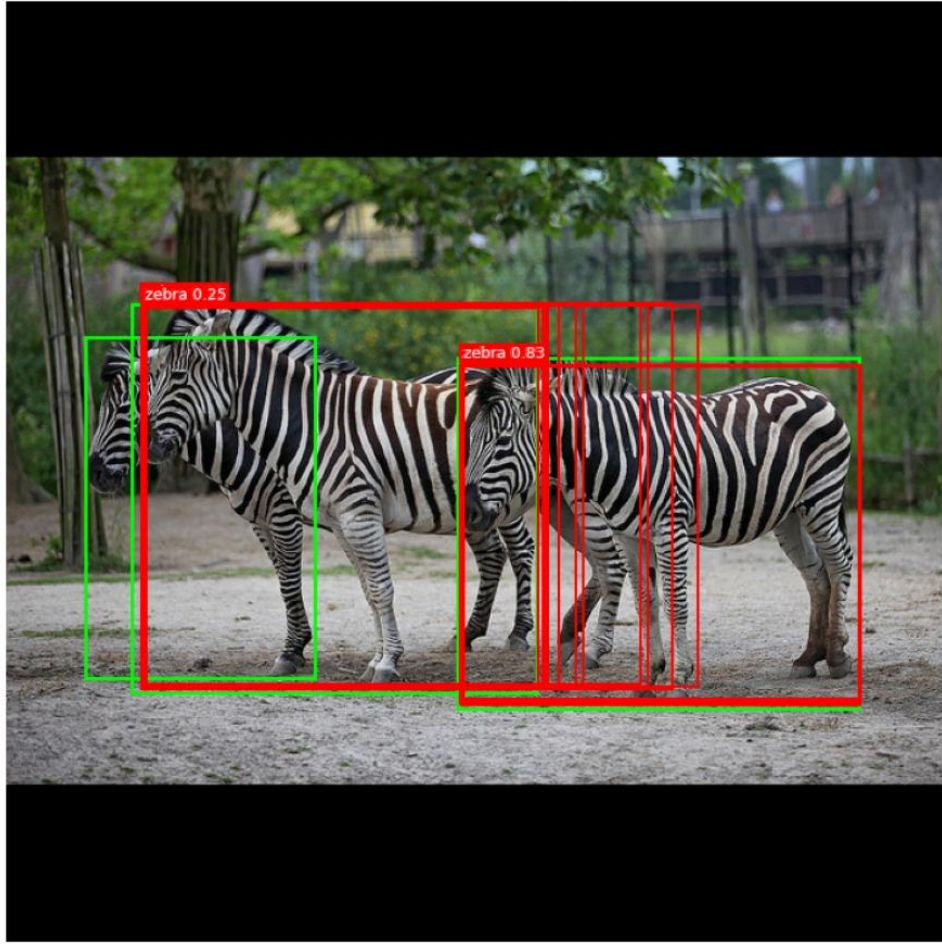
Detections after NMS: 6




```
plt.axis('off')  
plt.show()
```

```
plot_final_with_names(model, val_stream, device)
```

Inference Mode | Green=GT | Red=Pred



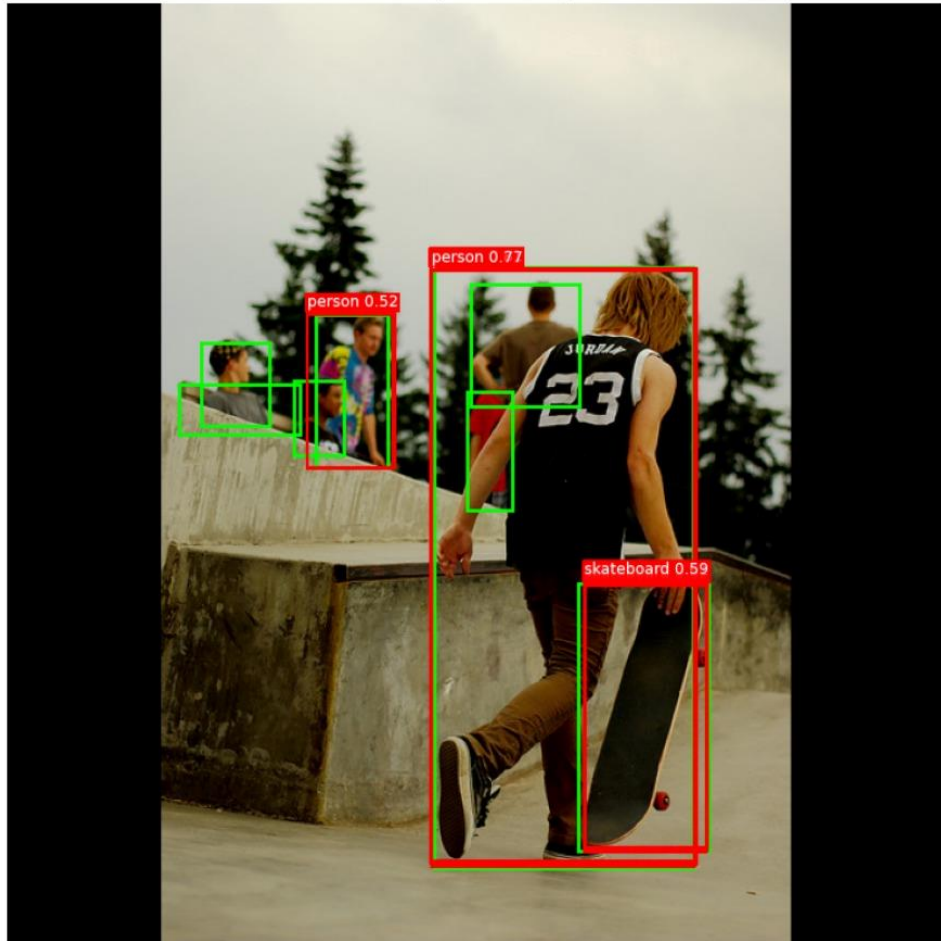
```
[ ]: import os
```



```
plt.axis('off')  
plt.show()
```

```
plot_final_with_names(model, val_stream, device)
```

Inference Mode | Green=GT | Red=Pred



```
[ ]: import os
```

Discussion

The use of the SiLU activation function provided smoother gradients compared to standard ReLU. The C2f blocks allowed for a deeper network without a proportional increase in parameter count, which is critical for maintaining the target inference speed. This is a small model which is trained on less dataset that is why it is a little faulty and is trained on almost 100 epochs so the output is not very good, a large model and more training will be suitable for better results

Challenges and Limitations

One significant challenge was the management of checkpoint files. During development, directory-style saves (e.g., data.pkl) caused loading conflicts with standard torch.load calls. Furthermore, the limited size of the dataset, while excellent for debugging, requires scaling to the full set for comprehensive performance metrics.

Ethical Considerations

Object detection systems must be deployed with a focus on privacy. Data used in this project is public-domain, but real-world applications must ensure that surveillance data is handled according to local privacy laws and that model biases are audited to prevent discriminatory misclassification.

Applications and Use Cases

- **Smart Surveillance:** Detecting unauthorized entry in restricted areas.
- **Traffic Monitoring:** Real-time counting of cars, buses, and trucks.
- **Retail Analytics:** Monitoring customer movement and shelf stock levels.

Conclusion

The project successfully implemented a modular and optimized object detection system. By focusing on architectural efficiency and numerical stability through mixed-precision techniques, the system demonstrates the feasibility of high-speed, multi-class detection in a PyTorch environment.

Future Work

Future iterations will focus on:

1. Scaling training to the full large scale dataset.
2. Exporting the model to TensorRT or ONNX for even lower latency on edge devices.
3. Implementing a real-time webcam inference script on CPU

References

1. Redmon, J., et al. "You Only Look Once: Unified, Real-Time Object Detection."
2. PyTorch Documentation (torch.nn, torch.amp).
3. Ultralytics YOLOv8 Architecture Overview.
4. COCO Dataset (Common Objects in Context) - cocodataset.org.